

Лабораторная работа №3

Построение многослойной нейронной сети

Для получения более высокой точности мы должны предоставить нейронной сети большую свободу. Это означает, что нейронная сеть должна иметь более одного слоя для извлечения базовых закономерностей, существующих среди тестовых данных. Давайте создадим многослойную нейронную сеть, которая нам это обеспечит.

Создайте новый файл Python и импортируйте следующие файлы.

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

В двух предыдущих разделах было продемонстрировано использование нейронной сети в качестве классификатора. В этом разделе будет показано, как использовать нейронную сеть в качестве регрессора. Сгенерируем выборочный набор точек данных, используя уравнение $y = 3x^2 + 5$, а затем нормализуем данные.

```
# Генерация тренировочных данных
min_val = -15
max_val = 15
num_points = 130
x = np.linspace(min_val, max_val, num_points)
y = 3 * np.square(x) + 5
y /= np.linalg.norm(y)
```

Переформируем приведенные выше переменные для создания тренировочного набора данных.

```
# Создание данных и меток
data = x.reshape(num_points, 1)
labels = y.reshape(num_points, 1)
```

Построим график входных данных.

```
# Построение графика входных данных
plt.figure()
plt.scatter(data, labels)
plt.xlabel('Размерность 1')
plt.ylabel('Размерность 2')
plt.title('Входные данные')
```

Определим многослойную нейронную сеть с двумя скрытыми слоями. Вы можете спроектировать нейронную сеть любым другим желаемым способом.

В данном случае у нас будет 10 нейронов в первом слое и 6 нейронов во втором слое. Наша задача заключается в предсказании одного значения, поэтому выходной слой будет содержать всего один нейрон.

```
# Определение многослойной нейронной сети с двумя скрытыми
# слоями. Первый скрытый слой состоит из десяти нейронов.
# Второй скрытый слой состоит из шести нейронов.
# Выходной слой состоит из одного нейрона.
nn = nl.net.newff([[min_val, max_val]], [10, 6, 1])
```

Установим метод градиентного спуска в качестве обучающего алгоритма.

```
# Задание градиентного спуска в качестве обучающего алгоритма
nn.trainf = nl.train.train_gd
```

Обучим нейронную сеть, используя сгенерированный ранее тренировочный набор данных.

```
# Тренировка нейронной сети
error_progress = nn.train(data, labels, epochs=2000, show=100, goal=0.01)
```

Запустим нейронную сеть для тренировочных точек данных.

```
# Выполнение нейронной сети на тренировочных данных
output = nn.sim(data)
y_pred = output.reshape(num_points)
```

Построим график продвижения процесса обучения.

```
# Построение графика ошибки обучения
plt.figure()
plt.plot(error_progress)
plt.xlabel('Количество эпох')
plt.ylabel('Ошибка обучения')
plt.title('Изменение ошибки обучения')
```

Построим график предсказанных результатов.

```
# Построение графика результатов
x_dense = np.linspace(min_val, max_val, num_points * 2)
y_dense_pred =
nn.sim(x_dense.reshape(x_dense.size,1)).reshape(x_dense.size)

plt.figure()
plt.plot(x_dense, y_dense_pred, '-', x, y, '.', x, y_pred, 'p')
plt.title('Фактические и прогнозные значения')

plt.show()
```

В процессе выполнения этого кода на экране отобразятся три графика. На первом снимке экрана представлены входные данные (рис. 14.6).

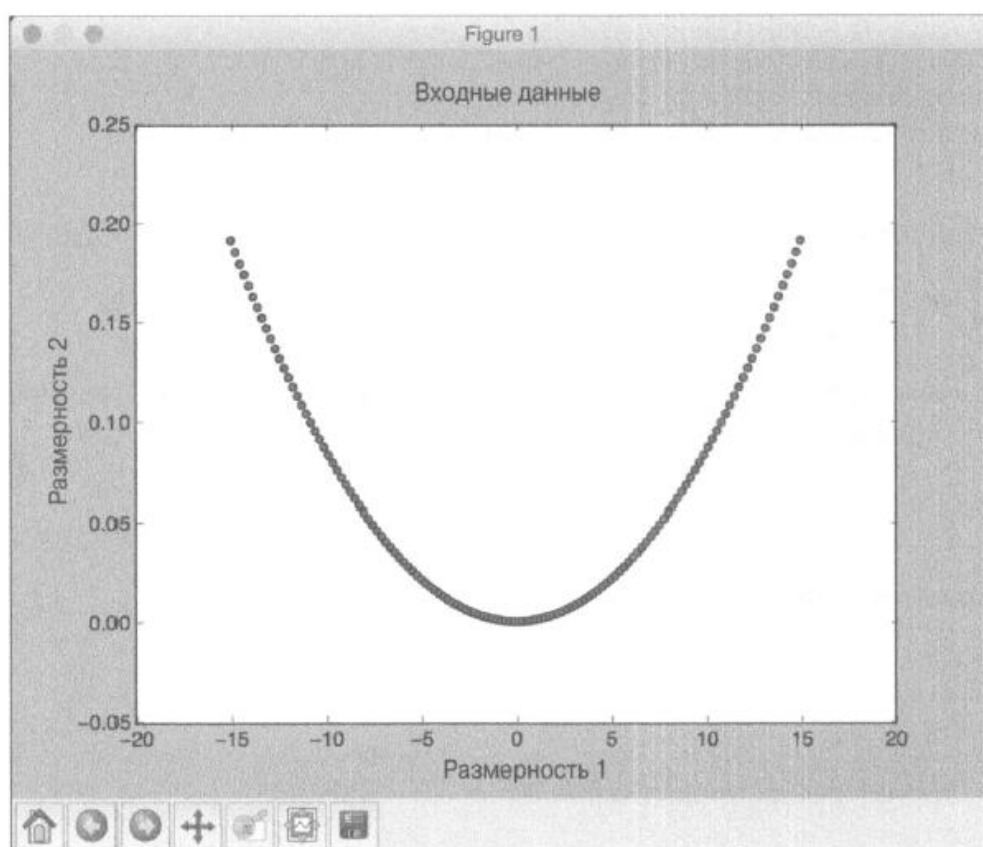


Рис. 14.6

На втором снимке экрана представлен график продвижения процесса обучения (рис. 14.7).

На третьем снимке экрана представлен график предсказанных результатов, наложенный на график входных данных (рис. 14.8).

Как видим, предсказанные результаты следуют общей тенденции. Если вы продолжите обучение сети и уменьшите ошибку, то увидите, что соответствие предсказанных результатов кривой исходных данных будет характеризоваться еще более высокой точностью.

В окне терминала отобразится следующая информация (рис. 14.9).

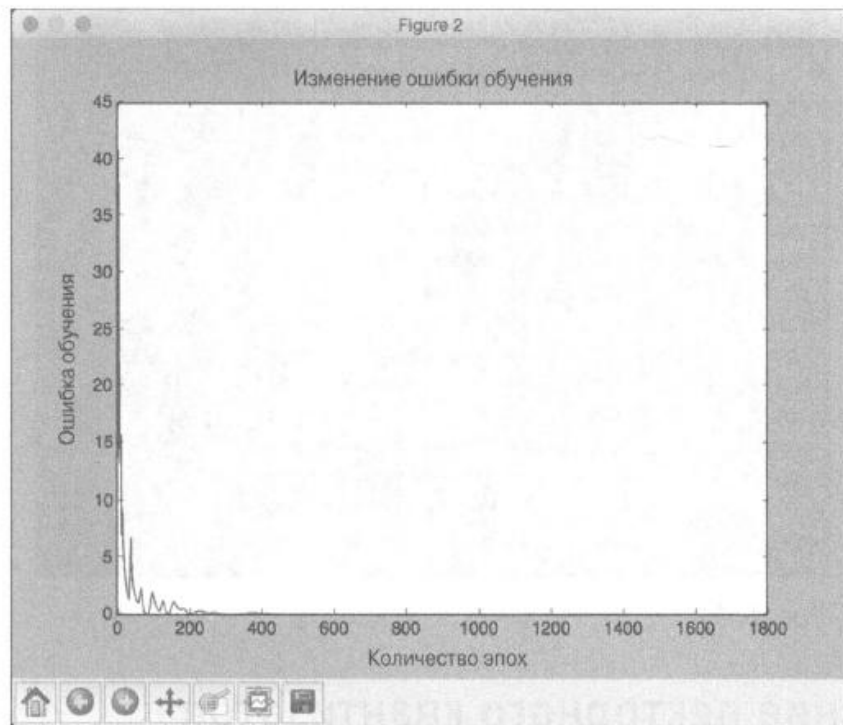


Рис. 14.7

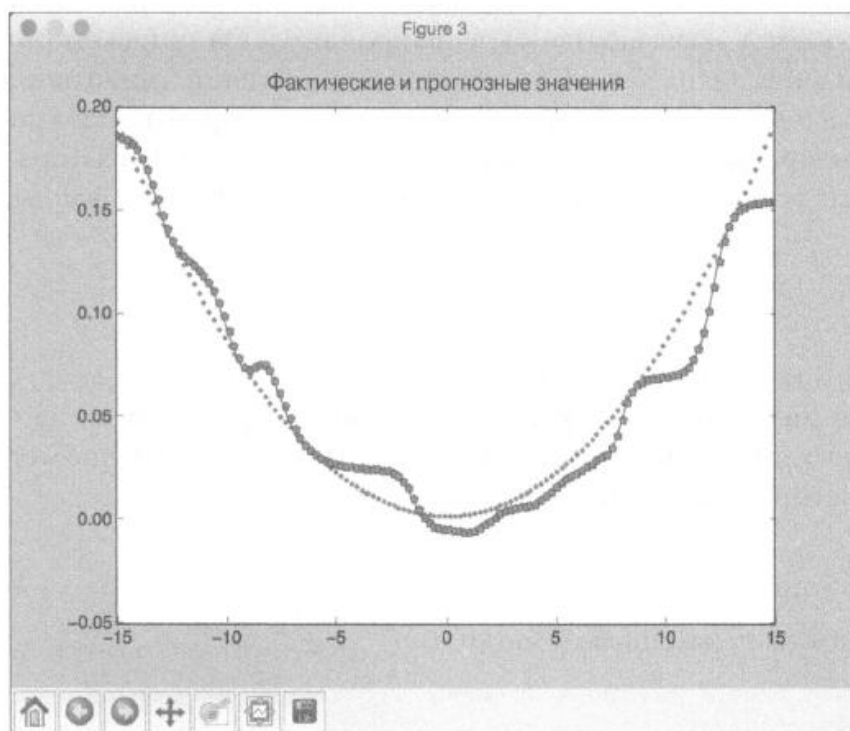


Рис. 14.8

```
Epoch: 100; Error: 1.9247718251621995;  
Epoch: 200; Error: 0.15723294798079526;  
Epoch: 300; Error: 0.021680213116912858;  
Epoch: 400; Error: 0.1381761995539017;  
Epoch: 500; Error: 0.04392553381948737;  
Epoch: 600; Error: 0.02975401597014979;  
Epoch: 700; Error: 0.014228560930227126;  
Epoch: 800; Error: 0.03460207842970052;  
Epoch: 900; Error: 0.035934053149433196;  
Epoch: 1000; Error: 0.025833284445815966;  
Epoch: 1100; Error: 0.013672412879982398;  
Epoch: 1200; Error: 0.01776586425692384;  
Epoch: 1300; Error: 0.04310242610384976;  
Epoch: 1400; Error: 0.03799681296096611;  
Epoch: 1500; Error: 0.02467030041520845;  
Epoch: 1600; Error: 0.010094873168855236;  
Epoch: 1700; Error: 0.01210866043021068;  
The goal of learning is reached
```

Рис. 14.9

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. Что называют многослойной нейронной сетью?
2. В чем заключается метод градиентного спуска?
3. Как происходит обучение многослойной нейронной сети?
4. Привести код на Python модели многослойной нейронной сети, рассмотренной в методических указаниях.
5. Самостоятельно провести эксперименты с многослойной нейронной сетью сгенерировав выборочный набор точек данных, используя непрерывную произвольную функцию $y(x)$. Представить график продвижения процесса обучения, а также график фактических и прогнозных значений.

```

import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl

# Generate some training data
min_val = -15
max_val = 15
num_points = 130
x = np.linspace(min_val, max_val, num_points)
y = 3 * np.square(x) + 5
y /= np.linalg.norm(y)

# Create data and labels
data = x.reshape(num_points, 1)
labels = y.reshape(num_points, 1)

# Plot input data
plt.figure()
plt.scatter(data, labels)
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Input data')

# Define a multilayer neural network with 2 hidden layers;
# First hidden layer consists of 10 neurons
# Second hidden layer consists of 6 neurons
# Output layer consists of 1 neuron
nn = nl.net.newff([[min_val, max_val]], [10, 6, 1])

# Set the training algorithm to gradient descent
nn.trainf = nl.train.train_gd

# Train the neural network
error_progress = nn.train(data, labels, epochs=2000, show=100, goal=0.01)

# Run the neural network on training datapoints
output = nn.sim(data)
y_pred = output.reshape(num_points)

# Plot training error
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Error')
plt.title('Training error progress')

# Plot the output
x_dense = np.linspace(min_val, max_val, num_points * 2)
y_dense_pred = nn.sim(x_dense.reshape(x_dense.size,1)).reshape(x_dense.size)

plt.figure()
plt.plot(x_dense, y_dense_pred, '-', x, y, '.', x, y_pred, 'p')
plt.title('Actual vs predicted')

plt.show()

```

Рис. 14.П1. Пример кода модели многослойной нейронной сети на Python

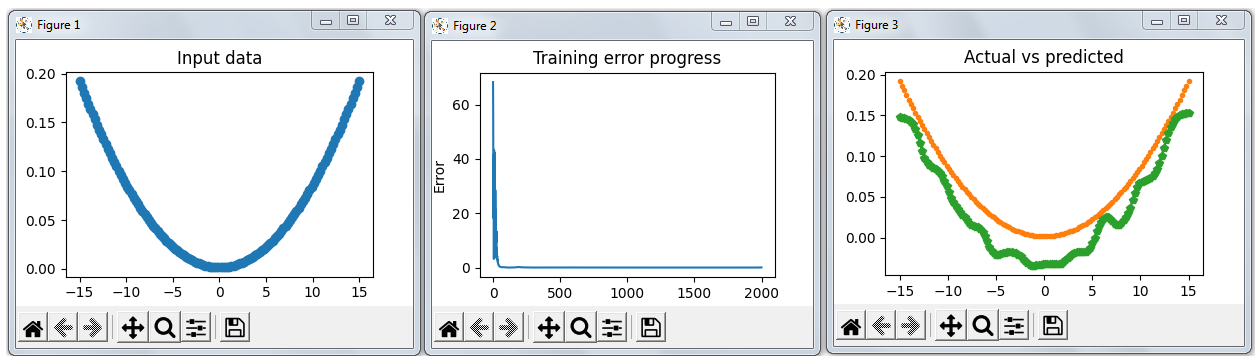


Рис. 14.П2. Входные точки данных, изменение ошибки обучения в зависимости от числа наблюдаемых эпох, график предсказанных результатов, наложенный на график исходных данных

```
Epoch: 100; Error: 0.05723027786317182;
Epoch: 200; Error: 0.18414073970894845;
Epoch: 300; Error: 0.031811420247265965;
Epoch: 400; Error: 0.06514596537769415;
Epoch: 500; Error: 0.04144240893156513;
Epoch: 600; Error: 0.046566898004469115;
Epoch: 700; Error: 0.03567999023168124;
Epoch: 800; Error: 0.03329818361661948;
Epoch: 900; Error: 0.027872231961336306;
Epoch: 1000; Error: 0.02526940971235741;
Epoch: 1100; Error: 0.022981980048591207;
Epoch: 1200; Error: 0.0210815179687619;
Epoch: 1300; Error: 0.01974382140707489;
Epoch: 1400; Error: 0.018587576788620053;
Epoch: 1500; Error: 0.017702594850295086;
Epoch: 1600; Error: 0.017066404846727938;
Epoch: 1700; Error: 0.01668812340580166;
Epoch: 1800; Error: 0.01582134159497378;
Epoch: 1900; Error: 0.055999215449760484;
Epoch: 2000; Error: 0.061258898893787106;
The maximum number of train epochs is reached
```

Рис. 14.П3. Результат выполнения кода